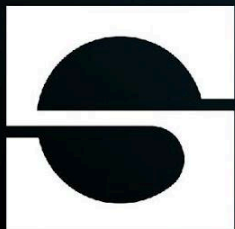




Saturn Dollar Stablecoin

Security Review



Disclaimer

Security Review

Saturn Dollar Yield bearing stablecoin



Disclaimer

The ensuing audit offers no assertions or assurances about the code's security. It cannot be deemed an adequate judgment of the contract's correctness on its own. The authors of this audit present it solely as an informational exercise, reporting the thorough research involved in the secure development of the intended contracts, and make no material claims or guarantees regarding the contract's post-deployment operation. The authors of this report disclaim all liability for all kinds of potential consequences of the contract's deployment or use. Due to the possibility of human error occurring during the code's manual review process, we advise the client team to commission several independent audits in addition to a public bug bounty program.

Table of Contents

Security Review

Saturn Dollar Yield bearing stablecoin



Table of Contents

Disclaimer	3
Summary	7
Scope	9
Methodology	9
Project Dashboard	13
Risk Section	16
Findings	18
3S-Stablecoin-H01	18
3S-Stablecoin-H02	20
3S-Stablecoin-M01	21
3S-Stablecoin-M02	23
3S-Stablecoin-M03	25
3S-Stablecoin-M04	27
3S-Stablecoin-M05	28
3S-Stablecoin-L01	30
3S-Stablecoin-L02	31
3S-Stablecoin-L03	33
3S-Stablecoin-L04	35
3S-Stablecoin-L05	36
3S-Stablecoin-L06	38
3S-Stablecoin-L07	40
3S-Stablecoin-L08	42
3S-Stablecoin-L09	44
3S-Stablecoin-L10	46
3S-Stablecoin-L11	48
3S-Stablecoin-L12	49
3S-Stablecoin-L13	51
3S-Stablecoin-N01	52
3S-Stablecoin-N02	54
3S-Stablecoin-N03	55

Summary

Security Review

Saturn Dollar Yield bearing stablecoin



Three Sigma audited Saturn Dollar in a 1.6 person week engagement. The audit was conducted from 30/12/2025 to 31/12/2025 and from 9/01/2026 to 12/01/2026.

Protocol Description

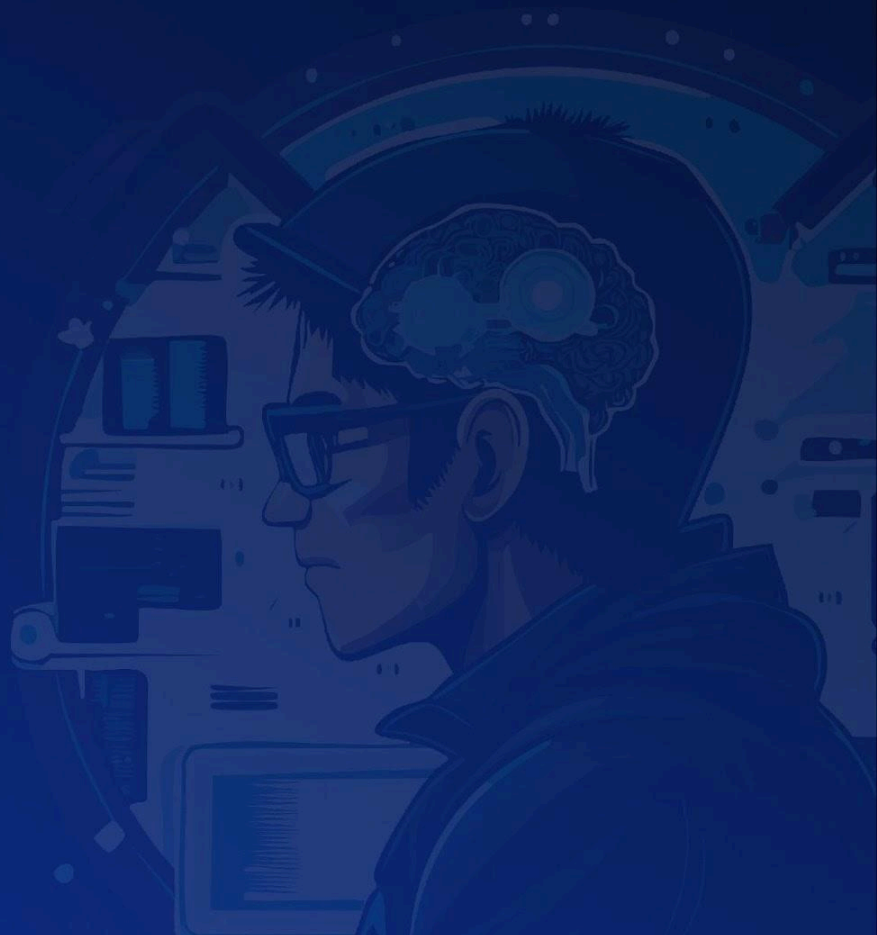
USDat is an upgradeable ERC20 token that implements role-based access control with three distinct privileges: administrators who manage upgrades, processors who mint new tokens, and compliance officers who manage the blacklist. The blacklist mechanism prevents flagged addresses from sending or receiving tokens and allows compliance to burn the balances of blacklisted accounts. All transfers and mints are checked against the blacklist before execution. The contract supports ERC20 Permit for signature-based approvals and includes a function to recover tokens mistakenly sent to the contract address.

Saturn Yield Dollar is a defi protocol that allows users to earn yield on their stablecoin holdings. Users deposit USDat tokens into the protocol and receive sUSDat shares in return, which automatically grow in value as rewards accumulate over time. The yield is generated through STRC, a treasury-backed reward token whose value is tracked via a price feed. The protocol includes administrative safeguards such as the ability to pause operations in emergencies and restrict certain addresses for regulatory compliance.

Scope

Security Review

Saturn Dollar Yield bearing stablecoin



Scope

Filepath	nSLOC
saturn-yield-dollar/src/StakedUSDat.sol	214
saturn-yield-dollar/src/WithdrawalQueue.sol	187
saturn-dollar/src/USDat.sol	80
saturn-yield-dollar/src/TokenizedSTRC.sol	51
Total	532

Methodology

Security Review

Saturn Dollar Yield bearing stablecoin



To begin, we reasoned meticulously about the contract's business logic, checking security-critical features to ensure that there were no gaps in the business logic and/or inconsistencies between the aforementioned logic and the implementation. Second, we thoroughly examined the code for known security flaws and attack vectors. Finally, we discussed the most catastrophic situations with the team and reasoned backwards to ensure they are not reachable in any unintentional form.

Taxonomy

In this audit, we classify findings based on ImmuneFi's [Vulnerability Severity Classification System \(v2.3\)](#) as a guideline. The final classification considers both the potential impact of an issue, as defined in the referenced system, and its likelihood of being exploited. The following table summarizes the general expected classification according to impact and likelihood; however, each issue will be evaluated on a case-by-case basis and may not strictly follow it.

Impact / Likelihood	LOW	MEDIUM	HIGH
NONE	None		
LOW	Low		
MEDIUM	Low	Medium	Medium
HIGH	Medium	High	High
CRITICAL	High	Critical	Critical

Project Dashboard

Security Review

Saturn Dollar Yield bearing stablecoin



Project Dashboard

Application Summary

Name	Saturn Dollar
Repository	R1: https://github.com/saturn-organization/saturn-dollar R2: https://github.com/saturn-organization/saturn-yield-dollar
Commit	R1 - ad2a465 R2 - aa762cc
Language	Solidity
Platform	Ethereum

Engagement Summary

Timeline	30/12/2025 to 31/12/2025 9/01/2026 to 12/01/2026
Nº of Auditors	2
Review Time	1.6 person weeks

Vulnerability Summary

Issue Classification	Found	Addressed	Acknowledged
Critical	0	0	0
High	2	2	0
Medium	5	4	1
Low	13	11	2
None	3	2	1

Category Breakdown

Suggestion	3
Documentation	1
Bug	18
Optimization	1
Good Code Practices	0

Risk Section

Security Review

Saturn Dollar Yield bearing stablecoin



Risk Section

No risks were identified.

Findings

Security Review

Saturn Dollar Yield bearing stablecoin



Findings

3S-Stablecoin-H01

Incomplete Blacklist Validation Allows Blacklisted Addresses to Transfer Tokens

Id	3S-Stablecoin-H01
Classification	High
Impact	High
Likelihood	High
Category	Bug
Status	Addressed in #d16820 .

Description

The USDat and StakedUSDat contracts implement a blacklist mechanism to freeze tokens held by non-compliant addresses. The **transfer()** and **transferFrom()** functions override the standard ERC20 implementations to enforce blacklist restrictions, but only validate the recipient address while ignoring the sender.

In **USDat::transfer()** and **StakedUSDat::transfer()**, the functions check if **to** is blacklisted but do not verify that **msg.sender** is not blacklisted. Similarly, **USDat::transferFrom()** and **StakedUSDat::transferFrom()** only validate the **to** address and do not check if **from** is blacklisted.

```
function transfer(address to, uint256 amount) public override returns (bool) {
    _requireNotBlacklisted(to);
    return super.transfer(to, amount);
}
function transferFrom(address from, address to, uint256 amount) public override returns (bool) {
    _requireNotBlacklisted(to);
    return super.transferFrom(from, to, amount);
}
```

This allows blacklisted addresses to transfer their tokens to non-blacklisted addresses, bypassing the freeze mechanism in both contracts.

Recommendation

Add blacklist validation for the sender address in both transfer functions across USDat and StakedUSDat contracts. In **transfer()**, check that **msg.sender** is not blacklisted. In **transferFrom()**, check that the **from** address is not blacklisted.

3S-Stablecoin-H02

Reward distribution front-running

Id	3S-Stablecoin-H02
Classification	High
Impact	High
Likelihood	Medium
Category	Bug
Status	Addressed in #8186d9e .

Description

The **transferInRewards** function mints rewards in a single batch operation:

```
function transferInRewards(uint256 amount) external nonReentrant
onlyRole(PROCESSOR_ROLE) notZero(amount) {
    TSTRC.mint(address(this), amount);
    emit RewardsReceived(amount);
}
```

According to the protocol documentation, yields from STRC and T-bills are used to purchase additional STRC for which tSTRC is subsequently minted. STRC distributes returns monthly with an annual yield of approximately 10%, corresponding to slightly under 1% per month. The issue is that this operation is executed in a single batch. A malicious user can make a large deposit immediately before the reward minting event and receive a disproportionately large share of the yield despite not being staked during the entire accrual period. After receiving the rewards, the user can immediately request withdrawal. The batch distribution model enables front-running attacks where users can time deposits to maximize rewards while minimizing actual stake duration.

Recommendation

Possible solutions:

- Gradual reward distribution using a streaming mechanism or vesting schedule
- Time-weighted reward calculation that accounts for stake duration

3S-Stablecoin-M01

Design flaws in **minUsdatReceived** slippage mechanism

Id	3S-Stablecoin-M01
Classification	Medium
Impact	Medium
Likelihood	Medium
Category	Bug
Status	Addressed in #7498148 .

Description

The current slippage protection via **minUsdatReceived** has design flaws that can lead to protocol losses and poor user experience.

1. Griefing attack vector

A malicious user can frontrun processRequests by calling **updateMinUsdatReceived** with an unrealistically high value after the off-chain STRC sale has already occurred:

```
// WithdrawalQueueERC721.sol:177-186
function updateMinUsdatReceived(uint256 tokenId, uint256 newMinUsdatReceived)
external {
    require(ownerOf(tokenId) == msg.sender, NotOwner());
    require(req.status == RequestStatus.Requested, AlreadyProcessed());
    req.minUsdatReceived = newMinUsdatReceived;
}
```

When processRequests executes, the validation fails:

```
// WithdrawalQueueERC721.sol:190-191
function _validateAmount(uint256 usdatAmount, uint256 minUsdatReceived) internal pure {
    require(usdatAmount >= minUsdatReceived, SlippageExceeded());
}
```

Since the off-chain sale has already occurred, the STRC is sold but the request cannot be finalized - resulting in a loss for the protocol. The minimum withdrawal of 10 USDat makes this attack cheap to execute.

2. Off-Chain synchronization issues

Even without malicious intent, frequent updates to **minUsdatReceived** create synchronization problems. The off-chain system may begin or partially completed a STRC sale based on one value of **minUsdatReceived**, while the user updates it at some point, causing the protocol to be unable to complete the entire withdrawal at the requested new price without incurring a loss for itself.

3. Variable share value during withdrawal period

Unlike the previous implementation, yield continues to accrue while a withdrawal request is pending. When a user creates a request, they set **minUsdatReceived** based on the current share value. However, during the withdrawal period (3-7 days), the share value increases due to vesting rewards.

Example:

1. User creates request for 100 shares at \$1.00/share, sets **minUsdatReceived** = 100
2. Over 5 days, yield accrues and share price rises to \$1.05
3. User's shares are now worth \$105, but their **minUsdatReceived** is still \$100
4. User receives \$100 instead of the \$105 their shares are now worth
5. To capture the full value, users would need to continuously update their **minUsdatReceived** throughout the withdrawal period - which is impractical and unlikely to happen. This creates a systematic disadvantage for users who don't actively monitor their pending requests.

Recommendation

One idea for a solution is to implement partial processing of withdrawal requests - when a portion of STRC is sold at a certain price, reflect this on-chain immediately. Changes to **minUsdatReceived** would only apply to the unprocessed portion. Additionally, limiting the frequency of **minUsdatReceived** updates (e.g., once per week) would provide predictability for off-chain operations.

3S-Stablecoin-M02

Price slippage risk in deposits and withdrawals

Id	3S-Stablecoin-M02
Classification	Medium
Impact	Medium
Likelihood	Medium
Category	Bug
Status	Addressed in #183bd15 .

Description

The **totalAssets** function in **StakedUSDat** depends on the STRC price from the oracle:

```
function totalAssets() public view override returns (uint256) {
    return IERC20(asset()).balanceOf(address(this)) + _strcTotalAssets();
}
function _strcTotalAssets() internal view returns (uint256) {
    (uint256 strcPrice, uint8 priceDecimals) = TSTRC.getPrice();
    uint256 strcBalance = TSTRC.balanceOf(address(this));

    return Math.mulDiv(strcBalance, strcPrice, 10 ** priceDecimals, Math.Rounding.Floor);
}
```

While the price of STRC is relatively stable, there have been periods of sharp price fluctuations. For example, in November 2025 the price dropped from \$97 to below \$90 for a day. During the same period there was cases in which the price changed by nearly 1% within 15 minutes.

1) Deposit slippage

A user submits a deposit transaction at one price, but by the time the transaction executes, the price may have moved significantly in an unfavorable direction. The user receives fewer shares than expected based on the price at transaction submission. With large amounts typically invested, this could result in substantial losses.

2) Withdrawal slippage

This problem is much more severe for withdrawals. When a user requests a withdrawal, the STRC amount is calculated based on the current price. However, the actual processing

occurs within 3–7 days. During this period, if the price drops, the user may receive significantly less USDat than expected (in the example from November 2025, the difference could exceed 5%).

Recommendation

Implement slippage protection for both deposits and withdrawals.

- 1) For deposits, allow users to specify a minimum number of shares they expect to receive and revert if the calculated shares are less.
- 2) The withdrawal problem is particularly challenging because part of the process happens off-chain. One way to address this is to let users set their slippage tolerance when they submit a withdrawal request. Then, during the off-chain execution of the STRC sale, the operator only processes requests that meet each user's tolerance, while the rest can be canceled during **processNext**. Alternatively, withdrawal requests could be made executable non-sequentially. This way, only the requests that pass the slippage check at a given time are executed, while the others wait for a better price. If a user decides to accept worse conditions, they should be able to cancel their existing request and create a new one with a different slippage tolerance.

3S-Stablecoin-M03

Incorrect share pricing during USDat depeg

Id	3S-Stablecoin-M03
Classification	Medium
Impact	High
Likelihood	Low
Category	Bug
Status	Acknowledged

Description

The **totalAssets** function assumes USDat maintains a 1:1 USD peg:

```
/// @notice ASSUMPTION: asset is USDat and is always 1 dollar backed by treasuries.
function totalAssets() public view override returns (uint256) {
    return IERC20(asset()).balanceOf(address(this)) + _strcTotalAssets();
}
function _strcTotalAssets() internal view returns (uint256) {
    (uint256 strcPrice, uint8 priceDecimals) = TSTRC.getPrice();
    uint256 strcBalance = TSTRC.balanceOf(address(this));

    return Math.mulDiv(strcBalance, strcPrice, 10 ** priceDecimals, Math.Rounding.Floor);
}
```

The function treats USDat balance as dollar-equivalent and directly adds it to the USD value of STRC holdings. If USDat loses its peg, this creates incorrect share pricing that enables arbitrage attacks. Example scenario:

- Contract holds 100,000 USDat + 5,000 tSTRC at \$100 each
- Protocol calculated totalAssets: 100,000 + 500,000 = 600,000
- USDat depegs to \$0.85 on the market
- Real USD value: 85,000 + 500,000 = 585,000
- But protocol still values it at 600,000

An arbitrageur can buy 100,000 USDat on the market for \$85,000 and deposit it into the protocol. The protocol values this deposit at \$100,000 and issues shares accordingly. The

arbitrageur instantly gains in share value at the expense of existing stakers. This exploits the discrepancy between market price and protocol valuation.

Recommendation

Implement a USDat price oracle to calculate the actual USD value of USDat holdings rather than assuming 1:1 peg.

Client Note: Instead of using STRC price in terms of dollars, We are planning to switch the oracle and use STRC price in terms of USDat. This limits to only supporting a single oracle although the math is slightly more complicated.

3S-Stablecoin-M04

Locked funds in **WithdrawalQueue** due to blacklisted users

Id	3S-Stablecoin-M04
Classification	Medium
Impact	Medium
Likelihood	Medium
Category	Bug
Status	Addressed in #06326e3 .

Description

The **USDat** token has a blacklist mechanism allowing user funds to be blocked. If a user submits a withdrawal request and is subsequently blacklisted, they will not be able to claim their tokens because the transfer will fail:

```
function claim(uint256 requestId) external nonReentrant whenNotPaused returns (uint256
amount) {
    Request storage req = queue[requestId];

    if (req.user != msg.sender) revert NotYourRequest();
    if (req.usdatOwed == 0 || req.claimed) revert NothingToClaim();

    req.claimed = true;
    amount = req.usdatOwed;

    IERC20(address(USDAT)).safeTransfer(msg.sender, amount); // Will fail if user is
blacklisted in USDat
}
```

The problem is that their **USDat** tokens will remain locked in the **WithdrawalQueue** contract with no mechanism to withdraw or burn them. The processed **USDat** sits permanently in the contract, reducing the circulating supply and creating an accounting discrepancy. There is no administrative function to recover or burn these locked tokens.

Recommendation

Implement an administrative function allowing authorized roles to seize or burn **USDat** from withdrawal requests belonging to blacklisted users.

3S-Stablecoin-M05

Withdrawal queue griefing attack

Id	3S-Stablecoin-M05
Classification	Medium
Impact	Medium
Likelihood	High
Category	Bug
Status	Addressed in #adf8ad2 .

Description

The **processNext** function processes batches of withdrawal requests sequentially in FIFO order. Requests within each batch are iterated through a **for** loop.2 times.

```
function processNext(uint256[] calldata usdatAmounts) external nonReentrant
onlyRole(PROCESSOR_ROLE) {
    uint256 count = usdatAmounts.length;
    if (count == 0) revert InvalidAmount();
    if (nextToProcess + count > queue.length) revert NoRequestsToProcess();
    uint256 totalUsdat = 0;
    uint256 totalStrc = 0;
    for (uint256 i = 0; i < count; i++) {
        totalUsdat += usdatAmounts[i];
        totalStrc += queue[nextToProcess + i].strcAmount;
    }
    USDAT.mint(address(this), totalUsdat);
    IERC20Burnable(address(TSTRC)).burn(totalStrc);
    for (uint256 i = 0; i < count; i++) {
        uint256 requestId = nextToProcess + i;
        Request storage req = queue[requestId];
        if (req.usdatOwed != 0) revert AlreadyProcessed();
        req.usdatOwed = usdatAmounts[i];
        emit WithdrawalProcessed(requestId, req.strcAmount, usdatAmounts[i]);
    }
    nextToProcess += count;
}
```

```
}
```

The **requestWithdraw** / **requestRedeem** functions only validates **notZero(assets)**, allowing requests as small as 1 wei. An attacker can spam thousands of minimal requests to grief the protocol. The processor must handle each request in sequential order before reaching legitimate requests further in the queue. Processing thousands of minimal withdrawal requests is significantly more expensive for the processor than creating them is for the attacker. This creates an unfavorable economic ratio where the protocol bears disproportionate gas costs. Legitimate user withdrawals can be delayed by days or weeks while spam requests are processed.

Recommendation

Implement a minimum withdrawal amount to prevent micro-withdrawals. Additional mitigations could include implementing rate limiting on withdrawal requests or introducing a cooldown period between consecutive requests.

3S-Stablecoin-L01

redistributeLockedAmount can be frontrun to claim unfair share of redistributed value

Id	3S-Stablecoin-L01
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Acknowledged

Description

When **redistributeLockedAmount** is called, the blacklisted user's shares are burned without reducing **totalAssets**, effectively distributing their value proportionally to all current shareholders. An attacker monitoring the mempool can frontrun this transaction by depositing assets, capturing part of the redistribution, then withdrawing for profit.

// StakedUSDat.sol:210-214

```
function redistributeLockedAmount(address from) external nonReentrant
onlyRole(DEFAULT_ADMIN_ROLE) {
    require(!_blacklisted[from], AddressNotBlacklisted());
    uint256 amountToDistribute = balanceOf(from);
    _burn(from, amountToDistribute);
}
```

The protocol already implements vesting for rewards via **transferInRewards** to prevent similar front-running attacks, but such protection is not applied to redistributed amounts. The impact is limited as it requires a large blacklisted account for meaningful profit.

Recommendation

A possible solution is to integrate **redistributeLockedAmount** with the vesting mechanism so the redistributed value is released gradually to shareholders rather than instantly.

Client note:

- 1- The solution would introduce more complex code
- 2- This functions is not likely to be called
- 3- A user who frontruns this function call, does not steal existing value from users but rather steals potentially new value that is not guaranteed to current holders.

3S-Stablecoin-L02

Withdrawal flow lacks direct path when vault holds **USDat** instead of **TSTRC**

Id	3S-Stablecoin-L02
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Acknowledged

Description

The withdrawal process always requires burning **TSTRC** via **burnQueuedShares**, even when the vault holds sufficient **USDat** to fulfill the withdrawal directly.

// StakedUSDat.sol:498-501

```
function burnQueuedShares(uint256 shares, uint256 strcAmount) external {
    require(msg.sender == address(WITHDRAWAL_QUEUE), OperationNotAllowed());
    IERC20Burnable(address(TSTRC)).burn(strcAmount);
    _burn(address(WITHDRAWAL_QUEUE), shares);
}
```

Consider the following scenario:

1. User deposits 100 **USDat**
2. Before conversion to **STRC** occurs, user requests withdrawal due to changing market conditions
3. The vault has sufficient **USDat** but no **TSTRC** to burn
4. **processRequests** will revert as there is no **TSTRC** to burn

One possible workaround would be for the processor to call **convertFromUsdat** to mint **TSTRC** without actually purchasing **STRC** off-chain. However, this would result in **TSTRC** minted on-chain without corresponding **STRC** held off-chain, potentially creating a compliance/regulatory issue regarding asset backing.

Additionally, if **TSTRC** from other users exists in the vault, it may be sold as part of the standard workflow - only to be re-purchased later - incurring unnecessary off-chain swap fees twice.

Recommendation

Implement an alternative withdrawal path that allows direct **USDat** withdrawals when the vault holds sufficient USDat balance, bypassing the **TSTRC** burn requirement.

Client note: We are planning to update the implementation in the future to allow for immediate withdrawals of USDat but this has larger design considerations.

3S-Stablecoin-L03

Hardcoded **PRICE_TOLERANCE_BPS** can permanently block withdrawals during extreme market conditions

Id	3S-Stablecoin-L03
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Addressed in #3495545 .

Description

PRICE_TOLERANCE_BPS is defined as an immutable constant set to 20%. During a black swan event where STRC price moves beyond this threshold, **processRequests** will revert, leaving withdrawal requests permanently stuck.

// WithdrawalQueueERC721.sol:66

uint256 public constant PRICE_TOLERANCE_BPS = 2000; // 20% tolerance

Example scenario:

1. User creates a withdrawal request for 1000 STRC
2. PROCESSOR sells STRC off-chain at \$100 and receives USDat
3. Market crashes, oracle now reports \$75 (25% drop)
4. PROCESSOR calls **processRequests** with **executionPrice** = \$100
5. Validation fails since \$100 is not within $\pm 20\%$ of \$75:

// WithdrawalQueueERC721.sol:227-229

(uint256 oraclePrice,) = TSTRC.getPrice();

require(!_isWithinTolerance(executionPrice, oraclePrice), OraclePriceMismatch());

The affected withdrawal requests become unprocessable as the off-chain sale has already occurred at the old price.

Recommendation

Replace the constant with an adjustable parameter controlled by admin.

3S-Stablecoin-L04

redistributeLockedAmount can lock assets permanently when no recipients exist

Id	3S-Stablecoin-L04
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Addressed in #e22eea4 .

Description

If a blacklisted user is the sole remaining shareholder and **StakedUSDat.redistributeLockedAmount** is called, the burned shares' value becomes attributed to ERC4626 virtual offset shares. These assets cannot be recovered since **rescueTokens** prohibits **USDat** and **TSTRC** withdrawal causing permanent loss. Additionally, future depositors will receive fewer shares than expected due to the inflated **totalAssets** / **totalSupply** ratio, losing value through ERC4626's round-down behavior on deposit.

```
function redistributeLockedAmount(address from) external nonReentrant
onlyRole(DEFAULT_ADMIN_ROLE) {
    require(!_blacklisted[from], AddressNotBlacklisted());
    uint256 amountToDistribute = balanceOf(from);
    _burn(from, amountToDistribute);
}
```

In practice, this can occur when **DEFAULT_ADMIN** and **PROCESSOR** submit transactions around the same time - **redistributeLockedAmount** to seize a blacklisted user's shares and **processRequests** to finalize all pending withdrawals. If **processRequests** gets included first, all non-blacklisted users exit the vault, leaving the blacklisted user as the sole shareholder before redistribution executes.

Recommendation

Add a check in **redistributeLockedAmount** to ensure recipients exist.

3S-Stablecoin-L05

Missing vested balance validation in `convertFromStrc` allows burning unvested rewards

Id	3S-Stablecoin-L05
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Addressed in #a42084 .

Description

The `StakedUSDat::convertFromStrc()` function only checks total tSTRC balance but does not validate that `strcAmount` being burned does not exceed the vested balance. This allows burning unvested tSTRC rewards, bypassing the vesting mechanism.

```
require(TSTRC.balanceOf(address(this)) >= strcAmount, InsufficientBalance());
// Missing check for vested balance
```

The contract tracks unvested amounts via `getUnvestedAmount()`, and `WithdrawalQueueERC721::_validateTotals()` correctly prevents selling unvested tSTRC, but `convertFromStrc()` lacks this protection.

Recommendation

Add vested balance validation before burning tSTRC, similar to `WithdrawalQueueERC721::_validateTotals()`.

```
function convertFromStrc(uint256 strcAmount, uint256 usdatAmount, uint256
strcSalePrice)
    external
    onlyRole(PROCESSOR_ROLE)
{
-   require(TSTRC.balanceOf(address(this)) >= strcAmount, InsufficientBalance());
+   uint256 strcBalance = TSTRC.balanceOf(address(this));
```

```
+ uint256 unvestedAmount = getUnvestedAmount();  
+ uint256 vestedBalance = strcBalance - unvestedAmount;  
+ require(strcAmount <= vestedBalance, InsufficientBalance());  
+  
  _validateConversion(usdatAmount, strcAmount, strcSalePrice);  
  // ...  
}
```

3S-Stablecoin-L06

Emitting contract balances instead of conversion amounts leads to incorrect event data

Id	3S-Stablecoin-L06
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Addressed in #197f81 .

Description

The **StakedUSDat::convertFromUsdat()** and **StakedUSDat::convertFromStrc()** functions emit the **Converted** event with contract balances instead of the actual conversion amounts passed as parameters. This prevents tracking specific conversion amounts from event logs since balances include other operations.

```
emit Converted(IERC20(asset()).balanceOf(address(this)),
TSTRC.balanceOf(address(this)));
```

The functions receive **usdatAmount** and **strcAmount** as parameters but emit total contract balances, making it impossible to determine the actual conversion amounts from events.

Recommendation

Emit the conversion amounts from function parameters instead of contract balances.

```
function convertFromUsdat(uint256 usdatAmount, uint256 strcAmount, uint256
strcPurchasePrice) {
    IERC20Burnable(asset()).burn(usdatAmount);
    TSTRC.mint(address(this), strcAmount);
-   emit Converted(IERC20(asset()).balanceOf(address(this)),
TSTRC.balanceOf(address(this)));
+   emit Converted(usdatAmount, strcAmount);
}
```

```
function convertFromStrc(uint256 strcAmount, uint256 usdatAmount, uint256
strcSalePrice) {
    IERC20Burnable(address(TSTRC)).burn(strcAmount);
    IUSDat(asset()).mint(address(this), usdatAmount);
-   emit Converted(IERC20(asset()).balanceOf(address(this)),
TSTRC.balanceOf(address(this)));
+   emit Converted(usdatAmount, strcAmount);
}
```


3S-Stablecoin-L07

Missing fee recipient validation in preview functions leads to inconsistent deposit calculations

Id	3S-Stablecoin-L07
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Addressed in #e238c9 .

Description

The `StakedUSDat::_deposit()` function applies fees only when both `depositFeeBps > 0` and `feeRecipient != address(0)`. However, `previewDeposit()` and `previewMint()` only check if `depositFeeBps == 0`, missing the `feeRecipient` validation. This causes preview functions to show fees being deducted when `depositFeeBps > 0` but `feeRecipient == address(0)`, while actual deposits execute without fees, leading to incorrect share calculations and potential slippage protection failures.

In `StakedUSDat::_deposit()`:

```
if (depositFeeBps > 0 && feeRecipient != address(0)) {
    fee = Math.mulDiv(assets, depositFeeBps, BPS_DENOMINATOR);
    IERC20(asset()).safeTransferFrom(caller, feeRecipient, fee);
}
```

But `previewDeposit()` only checks:

```
if (depositFeeBps == 0) {
    return super.previewDeposit(assets);
}
```

Recommendation

Add `feeRecipient` validation to both preview functions to match `_deposit()` logic.

```

function previewDeposit(uint256 assets) public view override returns (uint256) {
-   if (depositFeeBps == 0) {
+   if (depositFeeBps == 0 || feeRecipient == address(0)) {
        return super.previewDeposit(assets);
    }
    uint256 fee = Math.mulDiv(assets, depositFeeBps, BPS_DENOMINATOR);
    return super.previewDeposit(assets - fee);
}
function previewMint(uint256 shares) public view override returns (uint256) {
-   if (depositFeeBps == 0) {
+   if (depositFeeBps == 0 || feeRecipient == address(0)) {
        return super.previewMint(shares);
    }
    uint256 assets = super.previewMint(shares);
    return Math.mulDiv(assets, BPS_DENOMINATOR, BPS_DENOMINATOR -
depositFeeBps, Math.Rounding.Ceil);
}

```

3S-Stablecoin-L08

Lack of validation on **usdatAmounts**

Id	3S-Stablecoin-L08
Classification	Low
Impact	Medium
Likelihood	Low
Category	Bug
Status	Addressed in #183bd1 .

Description

The **WithdrawalQueue** contract processes withdrawal requests through **processNext()**, which accepts an array of **USDat** amounts from **PROCESSOR_ROLE** and assigns them to pending requests in FIFO order. The function lacks validation to ensure the provided amounts correspond to the correct requests.

When users call **requestWithdraw()** or **requestRedeem()** in **StakedUSDat**, the contract calculates **strcAmount** from the requested **USDat** amount using the current **STRC** price and stores it in the request. The original **USDat** amount is not preserved. Later, **PROCESSOR_ROLE** calculates **USDat** amounts from each request's **strcAmount** using the current **STRC** price at processing time:

```
function processNext(uint256[] calldata usdatAmounts) external nonReentrant
onlyRole(PROCESSOR_ROLE) {
    // ... validation checks ...

    for (uint256 i = 0; i < count; i++) {
        uint256 requestId = nextToProcess + i;
        Request storage req = queue[requestId];

        if (req.usdatOwed != 0) revert AlreadyProcessed();

        req.usdatOwed = usdatAmounts[i]; // No validation

        emit WithdrawalProcessed(requestId, req.strcAmount, usdatAmounts[i]);
    }
}
```

The function assigns **usdatAmounts[i]** directly to **queue[nextToProcess + i]** without validating that amounts are correct or in the correct order. If the processor provides amounts in the wrong order, miscalculates values, users receive incorrect USDat amounts.

Recommendation

Add some sort of validation on the **usdatAmounts** array in **processNext()** so that the order of the amounts should match the requestId orders

3S-Stablecoin-L09

Unbounded gas consumption in **_claimAll** function

Id	3S-Stablecoin-L09
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Addressed in #b6f2fcd .

Description

The **userRequestIds** array only accumulates entries and never removes them:

```
mapping(address => uint256[]) private userRequestIds;
function addRequest(address user, uint256 strcAmount) external {
    // ...
    userRequestIds[user].push(requestId); // Array grows indefinitely
}
function _claimAll(address user) internal returns (uint256 totalAmount) {
    uint256[] storage ids = userRequestIds[user];
    uint256 len = ids.length;

    for (uint256 i = 0; i < len; i++) { // Iterates over all entries
        Request storage req = queue[ids[i]];

        if (req.usdatOwed > 0 && !req.claimed) {
            req.claimed = true;
            totalAmount += req.usdatOwed;
            emit Claimed(ids[i], user, req.usdatOwed);
        }
    }
    // ...
}
```

The **_claimAll** function iterates over all entries every time it is called. As records accumulate over time for active users, gas costs increase linearly. An alternative claim function exists that allows claiming individual requests by ID, which mitigates the issue for users aware of

the gas implications. However, users relying on **claimAll** may experience unexpected high gas costs or transaction failures.

Recommendation

Implement a cleanup mechanism to remove claimed entries from the **userRequestIds** array or add pagination support to **claimAll** allowing users to specify a range of entries to process. Alternatively, document the gas cost implications and recommend users claim individual requests when they have accumulated many entries.

3S-Stablecoin-L10

Use of Revert Strings Instead of Custom Error

Id	3S-Stablecoin-L10
Classification	Low
Impact	Low
Likelihood	Low
Category	Optimization
Status	Addressed in #0fb1fe .

Description

The USDat and TokenizedSTRC contracts use revert strings in **require()** statements and **revert()** calls instead of custom errors. Revert strings are stored as bytecode, increasing deployment size and gas costs. When a revert occurs, the string must be encoded and returned, consuming additional gas. Custom errors only store a 4-byte selector, making them significantly cheaper.

```
require(!_blacklisted[account], "Recipient is blacklisted");
require(!_blacklisted[account], "Account is not blacklisted");
if (_blacklisted[account]) revert("Already blacklisted");
```

The contracts use Solidity version ^0.8.27, which supports using custom errors directly in **require()** statements (available from Solidity 0.8.26).

- Also make the below error more generic and change it to **Account is blacklisted**

```
function _requireNotBlacklisted(address account) internal view {
    require(!_blacklisted[account], "Recipient is blacklisted");
}
```

- Also consider to bump the Solidity version of StakedUSDat and WithdrawalQueue contracts to ^0.8.27

Recommendation

Replace revert strings with custom errors to reduce gas costs. Since the contracts use Solidity ^0.8.27, custom errors can be used directly in **require()** statements.

3S-Stablecoin-L11

Consider using named mappings

Id	3S-Stablecoin-L11
Classification	Low
Impact	Low
Likelihood	Low
Category	Documentation
Status	Addressed in #5a7880 .

Description

The USDat and StakedUSDat contracts use Solidity version ^0.8.27 and ^0.8.20, which supports named mappings introduced in Solidity 0.8.18. However, the contracts currently use unnamed mappings, such as **mapping(address => bool) private _blacklisted;**

mapping(address => bool) private _blacklisted;

Named mappings allow descriptive names for key and value types, making the mapping's purpose immediately clear without examining the variable name or surrounding code. This improves readability and reduces cognitive load when reviewing or maintaining the contract.

Recommendation

Use named mappings to improve code readability by explicitly naming the key and value types in the mapping declaration.

3S-Stablecoin-L12

Incorrect Parameter in ERC20Permit Initialization Leads to Invalid EIP-712 Domain Separator

Id	3S-Stablecoin-L12
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Addressed in #6ec269 .

Description

The **StakedUSDat** contract's **initialize()** function incorrectly passes the symbol to **__ERC20Permit_init()** instead of the token name. According to OpenZeppelin's documentation, **__ERC20Permit_init()** should receive the same name that is defined as the ERC-20 token name, as it initializes the EIP-712 domain separator which must be a user-readable name of the signing domain.

```
/**
 * @dev Initializes the {EIP712} domain separator using the `name` parameter, and setting
 * `version` to `1`.
 *
 * It's a good idea to use the same `name` that is defined as the ERC-20 token name.
 */
function __ERC20Permit_init(string memory name) internal onlyInitializing {
    __EIP712_init_unchained(name, "1");
}
```

When the symbol is passed instead of the name, the EIP-712 domain separator uses an incorrect domain name, causing permit signatures to be validated against the wrong domain and breaking compatibility with external systems that expect the domain name to match the token name.

Recommendation

Pass the token name to `__ERC20Permit_init()` instead of the symbol to ensure the EIP-712 domain separator uses the correct domain name.

3S-Stablecoin-L13

Missing oracle validation

Id	3S-Stablecoin-L13
Classification	Low
Impact	Medium
Likelihood	Low
Category	Bug
Status	Addressed in #6ad90b4 .

Description

The `getPrice` function retrieves the STRC price from the oracle without staleness or bounds validation:

```
function getPrice() external view returns (uint256 price, uint8 decimals) {
    require(address(oracle) != address(0), "Oracle not set");
    int256 answer = oracle.latestAnswer();
    require(answer > 0, "Invalid price from oracle");
    // forge-lint: disable-next-line(unsafe-typecast)
    price = uint256(answer);
    decimals = oracle.decimals();
}
```

There is no staleness check to verify when the price was last updated. Stale oracle data can persist indefinitely and be used for critical calculations in `totalAssets` and withdrawal conversions. There is also no bounds check to detect unrealistic price values that could result from oracle malfunction. Additionally, if this is a Chainlink oracle, the `latestAnswer` function is deprecated. Chainlink recommends using `latestRoundData` which returns additional metadata including the timestamp of the last update.

Recommendation

Replace **latestAnswer** with **latestRoundData** to access update timestamps. Implement staleness validation requiring price updates within a reasonable timeframe. Add minimum and maximum price bounds to detect anomalous values.

3S-Stablecoin-N01

convertFromUsdat can be frontrun to capture TVL Increase from favorable STRC purchase

Id	3S-Stablecoin-N01
Classification	None
Category	Suggestion
Status	Acknowledged

Description

When STRC is purchased off-chain at a price lower than the current oracle price, calling **convertFromUsdat** instantly increases the protocol's TVL. An attacker can frontrun this transaction to capture part of the profit.

```
// StakedUSDat.sol:319-332
function convertFromUsdat(uint256 usdatAmount, uint256 strcAmount, uint256
strcPurchasePrice)
    external
    onlyRole(PROCESSOR_ROLE)
{
    IERC20Burnable(asset()).burn(usdatAmount);
    TSTRC.mint(address(this), strcAmount);
}
```

The TVL is calculated using the oracle price not the purchase price:

```
function _strcTotalAssets() internal view returns (uint256) {
    (uint256 strcPrice, uint8 priceDecimals) = TSTRC.getPrice();
    // ...
    return Math.mulDiv(vestedBalance, strcPrice, 10 ** priceDecimals,
    Math.Rounding.Floor);
}
```

}

Example scenario:

1. PROCESSOR purchases 20,000 STRC off-chain at \$90 (cost: \$1,800,000)
2. Oracle price rises to \$100
3. Attacker deposits \$5,000,000 before **convertFromUsdat** executes
4. After execution: TVL increases by \$200,000 (20,000 × \$10 difference)
5. Attacker's share increases

The issue is that existing users' capital is used to purchase an asset that appreciates in value, while the attacker contributes nothing to the purchase yet claims a share of the profit. Unlike **transferInRewards**, the minted **TSTRC** from conversion is immediately vested and reflected in TVL. Currently, the attack is limited by the withdrawal queue delay (3-7 days), during which the price may move unfavorably. However, if instant withdrawals are implemented in the future, this would become a risk-free attack vector for value extract.

Recommendation

This issue is difficult to address with code changes alone. When executing conversions, care should be taken to minimize TVL displacement by managing the volume converted at once and the difference between the current oracle price and execution price.

Client note: We will plan to add this to our operations sheet so we can make sure to limit this attack vector as we use ConvertFromUSDat.

3S-Stablecoin-N02

Incorrect tolerance percentage in comments leads to misleading documentation

Id	3S-Stablecoin-N02
Classification	None
Category	Suggestion
Status	Addressed in #334795 .

Description

The `StakedUSDat::_validateConversion()` function uses `PRICE_TOLERANCE_BPS` constant set to 2000 (20% tolerance) for price validation, but the inline comments incorrectly state that validation is performed within $\pm 10\%$ of expected values. This documentation mismatch creates confusion about the actual tolerance limits enforced by the contract.

The constant is defined as:

```
uint256 public constant PRICE_TOLERANCE_BPS = 2000;
```

But the comments in `_validateConversion()` state:

```
// Validate strcAmount is within  $\pm 10\%$  of expected
require(_isWithinTolerance(strcAmount, expectedStrc), ExecutionPriceMismatch());
// Validate strcPurchasePrice against oracle price (within  $\pm 10\%$ )
require(_isWithinTolerance(strcPurchasePrice, oraclePrice), OraclePriceMismatch());
```

The actual tolerance enforced is 20%, not 10% as documented. This discrepancy misleads developers and auditors about the contract's price validation behavior.

Recommendation

Update the comments in `_validateConversion()` to reflect the actual 20% tolerance, or change `PRICE_TOLERANCE_BPS` to 1000 to match the documented 10% tolerance.

3S-Stablecoin-N03

Unreachable code in `_processWithdrawal` function

Id	3S-Stablecoin-N03
Classification	None
Category	Suggestion
Status	Addressed in #a048571 .

Description

The `_processWithdrawal` function contains an allowance check that is never executed:

```
function _processWithdrawal(address caller, address owner, uint256 assets, uint256
shares)
    internal
    nonReentrant
    notZero(assets)
    notZero(shares)
    returns (uint256 strcAmount)
{
    if (_blacklisted[caller] || _blacklisted[owner]) {
        revert OperationNotAllowed();
    }

    // ...

    if (caller != owner) {
        _spendAllowance(owner, caller, shares); // Never executed
    }

    _burn(owner, shares);
    // ...
}
```

This function is only called from `requestWithdraw` and `requestRedeem`, both of which pass `msg.sender` for both parameters:

```
function requestWithdraw(uint256 assets) external whenNotPaused {
    strcAmount = _processWithdrawal(msg.sender, msg.sender, assets, shares);
}
```

```
}  
function requestRedeem(uint256 shares) external whenNotPaused {  
    strcAmount = _processWithdrawal(msg.sender, msg.sender, assets, shares);  
}
```

Since caller and owner are always identical, the condition `caller != owner` is always false and the allowance spending code is never reached. This is dead code that wastes a small amount of gas on the conditional check and misleads readers into thinking an allowance mechanism exists for withdrawals.

Recommendation

Remove the unreachable allowance check from `_processWithdrawal` to improve code clarity and save gas. If allowance-based withdrawals are desired in the future, implement dedicated functions like `requestWithdrawFrom` and `requestRedeemFrom` that accept an owner parameter.